ECLIPSE ARTESANO — MANUAL DE CALIDAD DE SOFTWARE

Sistema e-Commerce de Artesanías hecho a mano | Código: MC-SQA-001

1. Control del Documento y Versiones

Nombre del Proyecto:	Eclipse Artesano (Plataforma Web E-Commerce)	**Código Documento:**	MC-SQA-001
Versión Actual:	1.0.0	**Fecha Elaboración:**	19/10/2026
Equipo / Autor(es):	Equipo de Desarrollo y SQA Eclipse Artesano	**Docente / Evaluador:**	Catedrático de Aseguramiento de Calidad

Historial de Revisiones

Versión	Fecha	Descripción del Cambio	Autor
0.1.0	10/10/2026	Borrador inicial de estándares, arquitectura web y roles para Eclipse Artesano.	Equipo SQA
1.0.0	19/10/2026	Aprobación oficial del Manual de Calidad para el sistema e-commerce de artesanías.	Equipo Lead SQA

2. Política y Objetivos de Calidad

2.1. Política de Calidad

En **Eclipse Artesano**, nos comprometemos a entregar una plataforma e-commerce web intuitiva, altamente responsiva, segura y libre de errores críticos, destinada a promover y comercializar artesanías únicas de colecciones tradicionales ("Manos de barro", "Legado Maya", "Casa de tejidos"). Garantizamos la excelencia técnica aplicando estándares rigurosos de arquitectura frontend (HTML5, CSS3 y JS ES6+), revisiones continuas de código y ejecución metódica de pruebas.

Nos enfocamos en mantener un código limpio, modular e higiénico que mantenga la consistencia funcional en la gestión interactiva del carrito de compras, cálculo de totales, modals dinámicos y navegación fluidas entre dispositivos móviles, tablets y escritorios.

2.2. Objetivos Medibles de Calidad

• **Cobertura de Pruebas:** Alcanzar un mínimo de 80% de cobertura de pruebas en módulos críticos (gestión del carrito de compras, cálculo de importes y eventos de navegación).

• **Densidad de Defectos:** Mantener una densidad inferior a 2 errores de severidad alta por cada 1,000 líneas de código fuente.

• **Tiempo de Respuesta (SLA):** Resolver cualquier fallo crítico detectado en producción en un tiempo no mayor a 12 horas.

• **Rendimiento y Responsividad:** Cumplimiento del 100% de usabilidad y adaptabilidad en resoluciones móviles (< 600px), tablet (< 900px) y escritorio.

3. Estructura Organizacional y Roles (Matriz RACI)

Fase / Entregable	Líder de Proyecto	Desarrollador Lead	Líder SQA / Tester
Definición de Requisitos y UI/UX	R / A	C	I
Construcción Frontend (HTML/CSS/JS)	I	R / A	C
Pruebas de Carrito y Layout Responsivo	I	C	R / A
Aprobación para Despliegue en Producción	A	R	R

Leyenda: R = Responsable, A = Aprobador, C = Consultado, I = Informado.

4. Estándares de Codificación (Coding Standards)

4.1. Reglas de Nomenclatura (Pila Web HTML/CSS/JavaScript)

• **Estructura HTML:** Utilizar semántica HTML5 (header, nav, section, aside, footer). Los identificadores id deben usar **camelCase** (ej. *cartPanel*, *listaCarrito*, *contador*). Atributos *alt* obligatorios en imágenes.

• **Estilos CSS:** Nombrado de clases mediante **kebab-case** explicativo (ej. *.cart-button*, *.hero-content*, *.section-title*). Separar bloques de código con comentarios descriptivos.

• **Funciones JavaScript:** Nombres en **camelCase** con verbos de acción (ej. *agregarCarrito()*, *actualizarCarrito()*, *vaciarCarrito()*, *abrirCarrito()*).

• **Variables y Constantes JS:** Variables dinámicas en **camelCase** (ej. *carrito*, *totalElemento*) y constantes globales en **UPPER_SNAKE_CASE**.

4.2. Ejemplo de Estructura de Código Conforme al Estándar

```
/* ========================================
   ECLIPSE ARTESANO - GESTIÓN DE CARRITO
   Estándar de Codificación JavaScript ES6
   ======================================== */

let carrito = []; // Estado global del carrito

/**
 * Agrega un producto artesanal al carrito de compras.
 * @param {string} nombre - Nombre del producto.
 * @param {number} precio - Precio unitario del producto.
 */
function agregarCarrito(nombre, precio = 0) {
    carrito.push({
        nombre: String(nombre).trim(),
        precio: Number(precio)
    });
    actualizarCarrito();
    abrirCarrito();
}

/**
 * Recalcula el monto total y actualiza el DOM del carrito.
 */
function actualizarCarrito() {
    const lista = document.getElementById("listaCarrito");
    const totalElemento = document.getElementById("total");
    let total = carrito.reduce((suma, prod) => suma + prod.precio, 0);

    totalElemento.innerText = "$" + total.toFixed(2);
}
```

5. Estrategia de Pruebas y Control de Errores (Bugs)

5.1. Clasificación de Severidad de Defectos

Severidad	Descripción del Error en Eclipse Artesano	SLA Máximo
Alta (Bloqueante)	Fallo crítico en el script del carrito de compras que impida agregar/eliminar ítems, cálculo incorrecto de montos o bloqueo total del modal.	Menos de 12 Horas
Media (Funcional)	El botón de cierre o la tecla ESC no cierran el carrito, desalineación de navegación en pantallas tablets o enlaces rotos a categorías.	Menos de 48 Horas
Baja (Cosmética)	Errores de ortografía en descripción de productos, desajustes menores de contraste o márgenes visuales en tarjetas de producto.	Próximo Sprint / Entrega

5.2. Flujo de Vida del Bug

Todo fallo detectado durante las pruebas unitarias, de interfaz o de usabilidad debe ser registrado en el gestor de tareas siguiendo el flujo:

[NUEVO] → [EN REVISIÓN] → [RESUELTO] → [VERIFICADO QA] → [CERRADO]

6. Criterios de Aceptación del Proyecto

Un módulo o versión del software **Eclipse Artesano** se considerará oficialmente **Aprobado** solo si cumple con la totalidad del siguiente checklist de verificación:

[ ]	El código HTML, CSS y JavaScript no presenta errores de sintaxis ni advertencias en consola.
[ ]	Cumple al 100% con los estándares de nomenclatura y estructura semántica estipulados.
[ ]	Las funciones del carrito (agregar, eliminar, vaciar, sumar total, modal y tecla ESC) han pasado las pruebas sin fallos.
[ ]	El diseño es 100% responsivo y funcional en resoluciones de escritorio, tablet y móvil.
[ ]	El código fuente ha sido revisado mediante *Code Review* por el Desarrollador Lead.
[ ]	No existen defectos pendientes con severidad Alta o Media.